

➤ ¿Qué es React DOM? ¿Cuáles son sus características y para qué sirve? Desarrollar un ejemplo sencillo en código REACT que permita visualizar lo descripto, explicando el ejemplo.

React-dom es la biblioteca de React encargada de conectar los componentes de la interfaz de usuario con el DOM (Modelo de Objetos del Documento) del navegador web.

Mientras que el paquete central react se encarga de definir los componentes, gestionar el estado y crear la representación virtual de la interfaz, react-dom sirve para traducir esas definiciones en elementos HTML reales y gestionar sus mutaciones en pantalla.

Características:

- Puede usarse en múltiples plataformas.
- No reescribe todo el HTML en cada cambio; solo toca los nodos específicos que cambiaron, reduciendo el costo de repintado y reflujo del navegador.
- Habilita el procesamiento concurrente, permitiendo al navegador pausar y priorizar renders sin congelar la pantalla.

Ejemplo:

Html:

```
<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="UTF-8" />
    <title>Ejemplo React DOM</title>
  </head>
  <body>
    <!-- Este es el contenedor destino donde React DOM montará la app -->
    <div id="root"></div>
  </body>
</html>
```

React:

```

import React, { useState } from 'react';
import ReactDOM from 'react-dom/client';

// Componente funcional simple
function Contador() {
  const [contador, setContador] = useState(0);

  return (
    <div style={{ fontFamily: 'sans-serif', padding: '20px' }}>
      <h2>Contador: {contador}</h2>
      <button onClick={() => setContador(contador + 1)}>
        Incrementar
      </button>
    </div>
  );
}

// 1. Obtenemos la referencia al nodo HTML real
const rootElement = document.getElementById('root');

// 2. React DOM crea la raíz concurrente
const root = ReactDOM.createRoot(rootElement);

// 3. React DOM renderiza el componente dentro del contenedor
root.render(<Contador />);

```

1. `document.getElementById('root')`: JavaScript estándar localiza el nodo `<div id="root">` dentro de la página HTML.
2. `ReactDOM.createRoot(...)`: React DOM toma control de ese nodo HTML y crea un punto de anclaje (*root*).
3. `root.render(<Contador/>)`: React DOM toma el árbol virtual creado por `<Contador/>`, genera los nodos reales (`<div>`, `<h2>`, `<button>`) y los inyecta en el navegador.
4. Ciclo de actualización: Cuando haces clic en el botón y `setContador` cambia el valor de 0 a 1, React calcula la diferencia entre el estado anterior y el nuevo. React DOM interviene únicamente para reemplazar el texto dentro del `<h2>`, sin tocar ni recrear el botón ni el contenedor.

➤ ¿Qué es React Router? ¿Cuáles son sus características y para qué sirve?
 Desarrollar un ejemplo sencillo en código REACT que permita visualizar lo descrito, explicando el ejemplo.

React Router es la librería estándar de enrutamiento para aplicaciones React en la web. Sirve para transformar una interfaz React en una Single Page

Application (SPA) con navegación dinámica, sincronizando la vista visible en pantalla con la URL del navegador sin necesidad de recargar la página entera desde el servidor.

Características:

- Las rutas se definen como componentes de React, integrándose de forma natural con la estructura del árbol de componentes.
- Permite renderizar sub-vistas dentro de una vista principal sin desmontar el contenedor padre.

Ejemplo:

```

import React from 'react';
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';

// 1. Componentes que representan las diferentes "páginas"
function Inicio() {
  return <h2>Página de Inicio: Bienvenido a nuestra app</h2>;
}

function AcercaDe() {
  return <h2>Acerca de: Somos un equipo apasionado por React</h2>;
}

function Contacto() {
  return <h2>Contacto: Escribenos a contacto@ejemplo.com</h2>;
}

function NoEncontrado() {
  return <h2>404: Página no encontrada</h2>;
}

// 2. Componente principal con la configuración de rutas
export default function App() {
  return (
    <BrowserRouter>
      <div style={{ fontFamily: 'sans-serif', padding: '20px' }}>
        {/* Barra de navegación */}
        <nav style={{ display: 'flex', gap: '15px', marginBottom: '20px' }}>
          <Link to="/">Inicio</Link>
          <Link to="/acerca">Acerca de</Link>
          <Link to="/contacto">Contacto</Link>
        </nav>

        <hr />

        {/* Definición de las rutas */}
        <main style={{ marginTop: '20px' }}>
          <Routes>
            <Route path="/" element={Inicio} />
            <Route path="/acerca" element={AcercaDe} />
            <Route path="/contacto" element={Contacto} />
            {/* Ruta comodín para manejar enlaces rotos o errores 404 */}
            <Route path="*" element={NoEncontrado} />
          </Routes>
        </main>
      </div>
    </BrowserRouter>
  );
}

```

<BrowserRouter>: Envuelve a toda la aplicación. Es el componente proveedor que escucha y sincroniza los cambios en la barra de direcciones usando la API de historial de HTML5.

<Link to="...">: Reemplaza a las etiquetas estándar `<a href="...">`. En lugar de forzar una petición HTTP y recargar la página completa, intercepta el clic y le indica a React Router qué ruta debe cargarse.

<Routes>: Funciona como un contenedor inteligente. Evalúa la URL actual y busca cuál de sus componentes hijos (<Route>) coincide mejor para mostrar únicamente ese elemento.

<Route element="{<...>" path="..."/> } >: Asocia un patrón de URL (path) con un componente específico (element).

path="*": Captura cualquier dirección que no coincida con las rutas declaradas previamente, sirviendo como vista de error 404.

➤ Teniendo en cuenta el contenido teórico/práctico visto en clase, realizar el ejercicio identificado en el anexo “GUÍA AUDIOVISUAL” con el título React.